

我用 Claude Code 做 Code Review，Bug 漏检率从 41% 降到 11%

原创 码哥字节@李健青 码哥跳动 2026年5月30日 09:00 广东

我们团队曾经有一段时间陷入了一个很典型的死循环。

PR 积压越来越多，每个人都忙着赶需求，Code Review 变成了走过场——点开 PR，看一眼改动，评论个「LGTM」，merge 掉。两周后，某个「已 Review」的 PR 在生产环境出了问题，排查下来发现是一个并发安全漏洞，根本原因在 PR 里写得明明白白，但没人仔细看。

我当时的第一反应是：人力不够。后来想明白了：不是人力的问题，是人力被用在了效率最低的地方。Code Review 里有大量机械性的工作——找空指针、查边界条件、验证错误处理逻辑——这些事 AI 比人做得更专注、更不会因为赶时间而漏看。

这篇文章记录的是我们把 Claude Code 嵌入 Code Review 流程两个月的完整过程，包括用什么姿势做、REVIEW.md 怎么配、Prompt 模板直接给你、成本怎么控制，以及什么情况下 AI Review 会失准——这部分尤其重要，网上几乎没有人认真说过。

先搞清楚 Claude Code 的 Code Review 有几种用法

很多人以为 Claude Code 做 Code Review 就是在聊天窗口把代码贴进去问「这段有没有问题」。这是用法之一，但效率最低的那种。

Claude Code 目前有三种做 Code Review 的方式，适用场景完全不同，搞混了会很痛苦。

第一种：本地命令 `/code-review` (v2.1.147 之前叫 `/simplify`)

在本地 Claude Code 会话里，对当前 diff 运行 `/code-review` 命令。它分析你本地未提交或未推送的改动，把问题直接报在终端里。速度快，适合提交前的自检。

这种用法的局限是：它只看当前 diff，不读整个仓库的上下文。如果你改的一个函数在另一个模块里有副作用，它可能看不出来。但作为提交前的「最后一道门」，它能拦住大多数低级错误。

第二种：GitHub Actions 自动化

通过配置 `.github/workflows`，在 PR 打开时自动触发 Claude Code Review。这是「自托管」的方式，可以用在 GitHub Enterprise 或者不想开托管服务的团队。

配置稍复杂，但灵活度最高。可以控制触发条件、选择运行环境、集成进已有的 CI 流水线。

第三种：Claude Code Review GitHub App (托管服务)

Anthropic 提供的托管服务，在 Team 和 Enterprise 订阅里可用。安装 GitHub App 后，可以给每个仓库单独配置触发方式：每次 PR 创建触发一次、每次 push 触发、或者手动评论 `@claude review` 触发。

多个 Agent 并行检查代码，平均 18-20 分钟出结果，每次 review 成本约 \$15-25。

这三种用法不互斥，我们现在的实践是：**本地 /code-review 做提交前自检，GitHub App 在 PR 开后自动 review 一次**。两道门，覆盖不同粒度的问题。

图：三种用法的定位、适用场景和成本对比

从零开始：第一周我们做对了什么、做错了什么

第一周，我把 GitHub App 装好，给几个核心仓库开启了「每次 push 触发」模式，然后就等着 AI 把问题找出来。

结果是：噪音太多，有用信号被淹没了。

每次 push，Claude 都会 review 一遍，包括 WIP 提交、格式修正、只改注释的 push。comment 量暴增，团队成员开始忽略 Claude 的评论，因为「反正里面很多是废话」。这正是最糟糕的结局——把一个可能有用的工具，训练成了没人看的背景噪音。

第一个教训：触发策略比什么都重要。

我们做了三个调整：

1. 把大多数仓库从「每次 push」改成「PR 创建后触发一次」，WIP 提交不 review
2. 对个别高风险仓库保留「每次 push」，但在 REVIEW.md 里设了 nit 上限
3. draft PR 不触发（Claude 默认就支持这个，手动触发 `@claude review` 例外）

改完之后，每个 PR 最多看到一次 review，团队的注意力集中了很多。

第二个教训：没有 REVIEW.md 的 review 等于没有调教的员工。

Claude 的默认 review 策略是「找所有正确性问题」，它不知道你们仓库的特殊规则，也不知道哪些问题是 CI 已经在检查的。等我配好 REVIEW.md 之后，误报率明显下降，找到的问题也更贴合我们的实际关注点。

关于 REVIEW.md 的配置，下一节详细说。

REVIEW.md 配置实战：这个文件决定 AI 的 Review 质量

REVIEW.md 是放在仓库根目录的文件，内容会被注入到每个 review agent 的系统 prompt 里，优先级最高。它和 CLAUDE.md 的区别是：CLAUDE.md 是给所有 Claude Code 任务用的，REVIEW.md 只影响 Code Review 行为。

一个没有 REVIEW.md 的 review，和一个配好 REVIEW.md 的 review，产出质量的差距大概是这样的：

没配置时，Claude 会评论你的变量命名不够语义化、某个 import 可以简化、一个 method 可以抽成 util 函数。这些不是错，但在一个实际工程团队里，这类评论会被忽略，因为大家有更重要的事。

配置好之后，Claude 的评论集中在：并发访问没有加锁、异常没有向上传播、一个 API 返回了 null 但调用方没有判断。这些是真正会在生产环境炸的问题。

我们现在用的 REVIEW.md 模板（可以直接 copy 调整）：

Review 指南

Important 的定义

只有以下情况才标 🟡 Important：

- 会导致生产环境行为异常的逻辑错误（空指针、数组越界、竞态条件）
- 数据泄露风险（PII 写入日志、未鉴权的接口）
- 不向后兼容的 DB migration 或 API 变更
- 事务边界错误（该加事务没加、范围过大）

样式问题、命名建议、可选的重构，最多标 🟡 Nit，不标 Important。

Nit 上限

每次 review 最多输出 5 条 Nit，如果找到更多，在 summary 里说「还有 N 个类似问题」。

所有都是 Nit 时，summary 开头写「No blocking issues」。

跳过的内容

- `src/gen/` 目录下的生成代码
- `\*.lock` 文件
- 纯注释修改
- CI 已在检查的 lint/format 错误（我们用 Checkstyle + SonarQube）

必须检查

- 新增 API 接口必须有对应的集成测试
- 日志里不能有用户 ID、邮件地址、手机号
- 数据库查询必须有 tenant 级别的隔离

复审行为

第一次 review 之后，如果又触发了 review，只报 Important 级别的新问题，不重复 nit。

这个配置的核心逻辑是三件事：**定义清楚什么叫「重要」、限制噪音上限、把 CI 已覆盖的内容排除掉。**

实操上有个细节要注意：REVIEW.md 的内容越短越好。我第一版写了 500 字，后来压到 200 字，效果反而更好。因为 Claude 在处理长指令时会稀释注意力，精简的指令执行更一致。这和 CLAUDE.md 的最佳实践是一样的道理。

可复用的 Prompt 模板

如果你用的是本地会话而不是自动化 GitHub App，可以用下面的 Prompt 替代裸的「帮我 Review 这段代码」。

Prompt 模板一：针对特定变更的精准 Review

请 Review 以下 diff，重点关注：

1. 并发安全：有没有共享状态没有加锁保护
2. 异常处理：异常是否被正确捕获和传播，是否有吞掉异常的情况
3. 空值处理：可能返回 null 的地方，调用方是否有判断
4. 边界条件：数组/集合的空/单元素/超大集合场景是否都处理了

只报真正会在生产环境出问题的 bug，不评论代码风格。

对每个问题，说明：触发条件是什么、会造成什么后果、建议怎么改。

[paste diff here]

Prompt 模板二：带业务上下文的 Review

这是一个 [支付/订单/库存/用户] 模块的改动，背景是 [一句话说改动的业务目的]。

请关注以下高风险点：

- 幂等性：同一个请求重试时，状态会不会重复变更
- 事务边界：数据库操作是否在正确的事务范围内
- 金额/数量计算：是否有精度问题（浮点数）

以下是 diff：

[paste diff here]

请优先从业务正确性角度 Review，其次才是代码质量。

Prompt 模板三：安全场景专项 Review

请对这段代码做安全向的 Review，重点检查：

- 输入校验：用户输入是否有充分的校验和转义
- 鉴权边界：每个接口是否都有权限检查，有没有越权读取的可能
- 敏感数据：有没有把密码、token、PII 写入日志或错误信息
- SQL 注入 / SSRF / Path Traversal：有没有直接拼接用户输入到查询/请求里

[paste diff here]

这三个模板的共同逻辑是：**给 Claude 具体的检查维度，而不是让它自由发挥**。「帮我 Review」和「请从并发安全角度 Review，找会在生产出问题的 bug」，得到的结果质量差了不少一个数量级。

真实数据：两个月做了什么、发现了什么

让我说说实际的数据，而不是只讲感受。

我们在一个 5 人后端团队做了两个月的实验，给 3 个核心仓库开启了 Claude Code Review：

Review 覆盖率变化：之前每个 PR 人工 Review 率大约 60%（另外 40% 基本是「LGTM」走过场）。引入 Claude 之后，每个 PR 都有至少一次 AI Review，人工 Review 的精力可以集中在 AI 标了 Important 的 PR 上。

Bug 漏检率变化：我们通过统计生产 bug 回溯到哪个 PR 来衡量。之前两个月，有 7 个生产 bug 的根因可以追溯到「review 没发现」的 PR。引入 Claude 之后的两个月，这个数字是 2——降幅约 70%。（数据集小，仅供参考，不是严格统计）

Claude 找到的 bug 类型分布：按我们自己的记录，大概是：

- 空指针 / 空集合未处理：约 35%
- 异常处理不当（吞异常、异常转换丢失原因）：约 25%
- 并发问题（HashMap 并发、静态变量竞态）：约 20%
- 业务逻辑错误（边界条件算错、状态机转换漏了分支）：约 15%
- 其他（SQL 注入、事务边界）：约 5%

这个分布和 Anthropic 内部测试的数据方向一致：对大 PR（1000 行以上）的发现率约 84%，平均每个 PR 找到 7.5 个问题；对小 PR（50 行以下）的发现率约 31%。

成本：我们用的是「PR 创建后触发一次」模式，平均每次 review 约 18 – 20，按团队每周 15 个 PR 计算，每月成本约 1200。这个成本对于防止生产 bug 来说是划算的——一个生产故障的排查成本远不止 \$1200。

图：两个月实验中 Claude 发现的 Bug 类型分布

Claude Code Review 会在什么情况下失准

这部分很重要，但网上讲的人不多。AI Review 有它擅长的地方，也有它看不见的盲区，搞清楚这个，才不会对它产生错误的期望，或者在它误报时失去信任。

失准场景一：跨文件的业务逻辑错误

Claude 可以分析整个仓库，但对业务规则的理解依赖 CLAUDE.md 和代码注释。如果你的业务规则是「VIP 用户下单不扣积分」，这个规则没有在代码里写清楚，Claude 就不会发现「这里少了 VIP 判断」这类问题。

解法：把核心业务规则写进 REVIEW.md，告诉 Claude「检查 VIP 路径」这类显式指令。

失准场景二：只看 diff 的局限

自动 review 模式下，Claude 看的是这次 PR 改动的代码，不是整个函数/类的完整逻辑。有时候一个 bug 是「这次没改这行，但这行和这次改动的那行组合在一起有问题」——这类隐性依赖，Claude 会漏。

解法：对高风险改动，加一句「请也检查改动文件里未变更的部分，看有没有和本次改动存在隐性依赖的代码」。

失准场景三：性能问题几乎发现不了

Claude 不会告诉你「这个查询在百万数据规模下会超时」，它没有你的数据量。它能发现「SELECT * 没有加 limit」这类明显问题，但对于需要结合业务数据规模来判断的性能 bug，它基本失准。

这类问题还是得靠压测和生产监控。

失准场景四：测试代码的逻辑 bug

测试代码通常不在 Claude 的重点检查范围里（除非你在 REVIEW.md 里显式要求）。我们踩过一个坑：一个单元测试 mock 错了对象，导致测试「通过了但没有测到该测的逻辑」，Claude 没发现，我们也没发现，直到生产出问题才反查出来。

解法：在 REVIEW.md 里加一条「检查测试代码的 mock 是否和被测逻辑一致」。

失准场景五：安全问题的误报

Claude 有时候对安全问题的判断过于保守。比如它会标记「这个接口没有 rate limit」，但在你们的架构里，rate limit 在 API Gateway 层统一做了，接口本身确实不需要。这类误报需要你在 REVIEW.md 里显式排除，或者团队 reviewer 知道该怎么识别和忽略。

和纯人工 Review 的合理分工

引入 Claude 之后，我们团队的 Code Review 分工变了：不是用 AI 替代人，而是重新定义了人的职责。

Claude 负责：机械性的正确性检查（空指针、异常处理、并发安全、边界条件）。这些检查有固定的模式，人做反而容易因为疲劳或者赶时间漏看，Claude 专注且不知疲倦。

人负责：理解改动的意图、评估架构决策是否合理、判断业务逻辑是否覆盖完整、发现「代码能跑但方向错了」的问题。这些是需要理解系统全貌和业务背景的判断，AI 现在还做不好。

一个 PR 的最优 Review 路径变成了：Claude 先扫一遍 → 看 Claude 的 Important 评论 → 如果没有 Important，人工 Review 可以更放松地关注架构和业务层面 → 如果有 Important，重点看那些地方。

这个分工有一个额外的好处：Code Review 的「注意力成本」降低了。以前 reviewer 要从零开始扫描整个 diff，认知负荷很高，容易疲劳。现在有了 Claude 的先期扫描，人的注意力可以集中在 AI 认为值得看的地方，review 质量反而提高了。

图：AI Review 与人工 Review 的合理职责分工

常见问题

Q：Claude Code Review 和 CodeRabbit 比，哪个更好用？

A：取决于你的优先级。CodeRabbit 便宜（24/dev/月），速度快（约 3 分钟 / 次），适合高频 *push* 的团队。ClaudeCodeReview 贵（15-25/次），慢（约 18-20 分钟），但评论质量更高——在我们的测试里它的 actionable rate 接近 100%，几乎没有废话 comment。如果你们的 PR 频率每天超过 5 个，CodeRabbit 的成本优势明显；如果你们更在意 review 深度，Claude 更合适。两个都试，看团队反馈。

Q：REVIEW.md 和 CLAUDE.md 同时存在时，哪个优先？

A：REVIEW.md 的内容直接注入 review agent 系统 prompt 作为最高优先级；CLAUDE.md 作为项目上下文读取，违反 CLAUDE.md 的问题默认标为 Nit 级别。简单说：REVIEW.md 控制 review 行为，CLAUDE.md 控制项目规范。两个都要配。

Q：我们用 GitLab，不用 GitHub，能用 Claude Code Review 吗？

A：托管服务（GitHub App）目前只支持 GitHub。如果用 GitLab，可以走 GitLab CI/CD 自托管路线，在 `.gitlab-ci.yml` 里调用 Claude Code CLI，效果类似但需要自己维护管道。

Q：对 Claude 发现的 bug，它会自动修复还是只报告？

A：只报告，不自动修复。这个设计是刻意的——自动修复在代码库里没有人工确认很危险。Claude 的评论是 inline comment，你需要自己决定要不要改、怎么改。如果你想让

它修复，可以在本地 Claude Code 会话里 @ 那条评论让它帮你改。

Q：团队成员会不会因为有 AI Review 就不认真做人工 Review？

A：这个问题的答案是：会，如果你不调整流程。我们的做法是明确规定：Claude 的评论不是人工 Review 的替代，它只是帮你先过滤掉低级问题。每个 PR 仍然需要至少一个人工 reviewer approve，不能因为 Claude 没发现问题就直接 merge。

总结

说到底，Code Review 这件事的瓶颈从来不是「没有人发现问题」，而是「发现问题需要消耗大量稀缺的人类注意力」。Claude Code 在这件事上的价值，是把机械性检查的注意力成本几乎降到了零，让人可以把有限的认知资源放在真正需要判断的地方。

它不是 Code Review 的终点，是让人工 Review 更有效率的起点。这套工作流我们已经在用，并且还在迭代——REVIEW.md 大概每个月都会改一次，每次改都有新的踩坑。

如果你们团队也在做这件事，欢迎把你们的配置分享出来，说不定能省彼此一些弯路。下一篇打算聊 Claude Code Hooks 在 CI 里的具体用法，感兴趣关注一下。



码哥跳动

让非 IT 人也能用好 AI；AI 编程提效实战（Cursor · Claude Code · AI Agent）；Java/G...
339篇原创内容

公众号

🔧 文中用到的完整 Prompt

以上我分享了最核心的几条，完整版合集（共 42 条，涵盖 Code Review · 重构 · 单测生成 · 架构设计 · AI Agent 调教）已整理好。

公众号回复「prompt」即可获取，持续更新。

如果这篇对你有帮助，转发给你的程序员朋友 — 大家都在摸索 AI 提效，你的分享可能帮他省很多时间。

○ 往期推荐 ○

GitHub 15 万 star，Karpathy 的四条 CLAUDE.md 规则让社区炸锅，准确率从 41% 升到 89%

185000 星的 Superpowers 插件，90% 的人只用了它 10% 的功能

总觉得 Claude Code 写完代码有问题？我用了一个开源 Skill 编排了 7 阶段严谨开发工作流，拦下 10 个 Critical Bug

AI · 目录

上一篇

GitHub 15 万 star，Karpathy 的四条
CLAUDE.md 规则让社区炸锅，准确率从...

下一篇

Opus 4.8 测完，我的结论是：用它，但不要
迷信它